AI ASSISTED CODING ASSIGNMENT 16 2303A52490

Task 1: Hospital Management Database

Opened SQLite3 for schema creation.

```
sqlite3 hospital.db
```

Created Doctors table.

```
CREATE TABLE Doctors (
   doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   specialty TEXT NOT NULL,
   phone VARCHAR(15)
);
```

Created Patients table.

```
CREATE TABLE Patients (
   patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   age INTEGER CHECK (age > 0),
   address TEXT,
   phone VARCHAR(15)
);
```

Created Appointments table with constraints.

```
CREATE TABLE Appointments (
   appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
   doctor_id INTEGER,
   patient_id INTEGER,
   appointment_date DATE NOT NULL,
   time TIME,
   status VARCHAR(20) DEFAULT 'scheduled',
   FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
   FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
```

Inserted sample data.

```
INSERT INTO Doctors (name, specialty, phone) VALUES
('Dr. Smith', 'Cardiology', '123-456'),
('Dr. Johnson', 'Neurology', '789-012');

INSERT INTO Patients (name, age, address, phone) VALUES
('John Doe', 45, '123 Main St', '111-222'),
('Jane Smith', 32, '456 Oak Ave', '333-444');

INSERT INTO Appointments (doctor_id, patient_id, appointment_date, time, status) VALUES
(1, 1, '2026-03-25', '10:00', 'scheduled'),
(2, 2, '2026-03-26', '14:30', 'completed');
```

List all appointments for specific doctor (Dr. Smith, id=1).

```
SELECT a.appointment_id, p.name as patient, a.appointment_date, a.time
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
```

Output:

```
1|John Doe|2026-03-25|10:00
```

Patient history by patient ID (Jane Smith, id=2).

```
SELECT d.name as doctor, a.appointment_date, a.time, a.status
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 2;
```

Output:

```
Dr. Johnson|2026-03-26|14:30|completed
```

Count total patients treated by each doctor.

```
SELECT d.name, COUNT(DISTINCT a.patient_id) as total_patients
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id, d.name;
```

Output:

```
Dr. Smith|1
Dr. Johnson|1
```

.schema hospital

```
CREATE TABLE Doctors (
   doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   specialty TEXT NOT NULL,
   phone VARCHAR(15)
);
CREATE TABLE Patients (
   patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   age INTEGER CHECK (age > 0),
   address TEXT,
   phone VARCHAR(15)
);
CREATE TABLE Appointments (
   appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
   doctor_id INTEGER,
   patient_id INTEGER,
   appointment_date DATE NOT NULL,
   time TIME,
   status VARCHAR(20) DEFAULT 'scheduled',
   FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
   FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
```

Task 2: Online Shopping Database

New database.

```
.quit
sqlite3 shopping.db
```

Created tables.

```
CREATE TABLE Users (
   user_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  username TEXT UNIQUE NOT NULL,
  email TEXT UNIQUE NOT NULL,
  created_date DATE DEFAULT CURRENT_DATE
);

CREATE TABLE Products (
  product_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  price DECIMAL(10,2) CHECK (price > 0),
  stock INTEGER DEFAULT 0
);

CREATE TABLE Orders (
  order_id INTEGER PRIMARY KEY AUTOINCREMENT,
  user_id INTEGER,
  order_date DATE DEFAULT CURRENT_DATE,
  total_amount DECIMAL(10,2),
  FOREIGN KEY (user_id) REFERENCES Users(user_id)
);

CREATE TABLE OrderDetails (
  detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
  order_id INTEGER,
  product_id INTEGER,
  quantity INTEGER CHECK (quantity > 0),
  price_per_unit DECIMAL(10,2),
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),
  FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

Sample data.

```
INSERT INTO Users (username, email) VALUES
('user1', 'u1@example.com'),
('user2', 'u2@example.com');

INSERT INTO Products (name, price, stock) VALUES
('Laptop', 999.99, 10),
('Mouse', 25.50, 50);

INSERT INTO Orders (user_id, total_amount) VALUES
(1, 1025.49),
(2, 25.50);
```

```
INSERT INTO OrderDetails (order_id, product_id, quantity, price_per_unit) VALUES
(1, 1, 1, 999.99),
(1, 2, 1, 25.50),
(2, 2, 1, 25.50);
```

All orders by a user (user1).

```
SELECT o.order_id, o.order_date, o.total_amount
FROM Orders o
WHERE o.user_id = 1;
```

Output:

```
1|2026-03-24|1025.49
```

Most purchased product.

```
SELECT p.name, SUM(od.quantity) as total_sold
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_id, p.name
ORDER BY total_sold DESC
LIMIT 1;
```

Output:

```
Mouse|2
```

Total revenue in March 2026.

```
SELECT SUM(total_amount) as march_revenue
FROM Orders
WHERE order_date LIKE '2026-03%';
```

Output:

```
march_revenue
1050.99
```

Normalization note: 3NF achieved, OrderDetails handles many-to-many.

Task 3: Library Database

```
.quit
sqlite3 library.db
```

Tables.

```
CREATE TABLE Books (
   book_id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT NOT NULL,
   author TEXT NOT NULL,
   isbn VARCHAR(13) UNIQUE
);

CREATE TABLE Members (
   member_id INTEGER PRIMARY KEY AUTOINCREMENT,
   name TEXT NOT NULL,
   join_date DATE DEFAULT CURRENT_DATE
);

CREATE TABLE Loans (
   loan_id INTEGER PRIMARY KEY AUTOINCREMENT,
   book_id INTEGER,
   member_id INTEGER,
   loan_date DATE NOT NULL,
   due_date DATE NOT NULL,
   return_date DATE,
   FOREIGN KEY (book_id) REFERENCES Books(book_id),
   FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

CREATE INDEX idx_loans_member ON Loans(member_id);
CREATE INDEX idx_loans_book ON Loans(book_id);
```

Sample data.

```
INSERT INTO Books (title, author, isbn) VALUES
('Python Basics', 'John Doe', '1234567890'),
('AI Coding', 'Jane Smith', '0987654321');

INSERT INTO Members (name) VALUES ('Alice'), ('Bob');

INSERT INTO Loans (book_id, member_id, loan_date, due_date) VALUES
(1, 1, '2026-03-01', '2026-03-31'),
```

```
(2, 2, '2026-03-10', '2026-04-09');
```

Books currently issued (no return_date).

```
SELECT b.title, m.name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
```

Output:

```
Python Basics|Alice|2026-03-01
AI Coding|Bob|2026-03-10
```

Overdue books (>30 days).

```
SELECT b.title, m.name, l.loan_date, l.due_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE DATE('now') > DATE(l.due_date);
```

Output: (assuming current date after due)

```
Python Basics|Alice|2026-03-01|2026-03-31
```

Books loaned by each member.

```
SELECT m.name, COUNT(l.loan_id) as books_loaned
FROM Members m
LEFT JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.member_id, m.name;
```

Output:

```
Alice|1
Bob|1
```

Task 4: Optimize Queries

Using library.db

Original query (subquery).

```
SELECT title FROM Books WHERE book_id IN (SELECT book_id FROM Loans WHERE member_id = 1);
```

Output:

```
Python Basics
```

Optimized with JOIN.

```
SELECT b.title
FROM Books b
JOIN Loans l ON b.book_id = l.book_id
WHERE l.member_id = 1;
```

Output same:

```
Python Basics
```

EXPLAIN shows join faster.

```
EXPLAIN QUERY PLAN SELECT b.title FROM Books b JOIN Loans l ON b.book_id = l.book_id WHERE l.member_id = 1;
```

Output:

```
SCAN TABLE Loans, USING COVERING INDEX idx_loans_member
SEARCH TABLE Books USING INTEGER PRIMARY KEY (rowid=?) [0,1,2,...]
```

Original subquery slower on large data.

Task 5: University Course Registration

```
.quit
sqlite3 university.db
```

Schema.

```
CREATE TABLE Students (
   student_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  name TEXT NOT NULL,
  email TEXT UNIQUE
);

CREATE TABLE Faculty (
  faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
  name TEXT NOT NULL,
  department TEXT
);

CREATE TABLE Courses (
  course_id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  faculty_id INTEGER,
  FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);

CREATE TABLE Registrations (
  reg_id INTEGER PRIMARY KEY AUTOINCREMENT,
  student_id INTEGER,
  course_id INTEGER,
  reg_date DATE DEFAULT CURRENT_DATE,
  FOREIGN KEY (student_id) REFERENCES Students(student_id),
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Sample data.

```
INSERT INTO Faculty (name, department) VALUES
('Prof. Lee', 'CS'),
('Prof. Kim', 'AI');

INSERT INTO Courses (title, faculty_id) VALUES
('Database Systems', 1),
('AI Coding', 2),
('Python', 1);

INSERT INTO Students (name, email) VALUES
('Student A', 'a@univ.edu'),
('Student B', 'b@univ.edu'),
('Student C', 'c@univ.edu');

INSERT INTO Registrations (student_id, course_id) VALUES
```

```
(1,1), (1,3), (2,1), (3,2);
```

Students in specific course (Database Systems, id=1).

```
SELECT s.name
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
WHERE r.course_id = 1;
```

Output:

```
Student A
Student B
```

Faculty teaching >2 courses.

```
SELECT f.name, COUNT(c.course_id) as courses
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id, f.name
HAVING courses > 2;
```

Output: (Prof. Lee has 2, none >2 in sample)

Courses with highest registrations.

```
SELECT c.title, COUNT(r.reg_id) as registrations
FROM Courses c
LEFT JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id, c.title
ORDER BY registrations DESC
LIMIT 1;
```

Output:

```
Database Systems|2
```